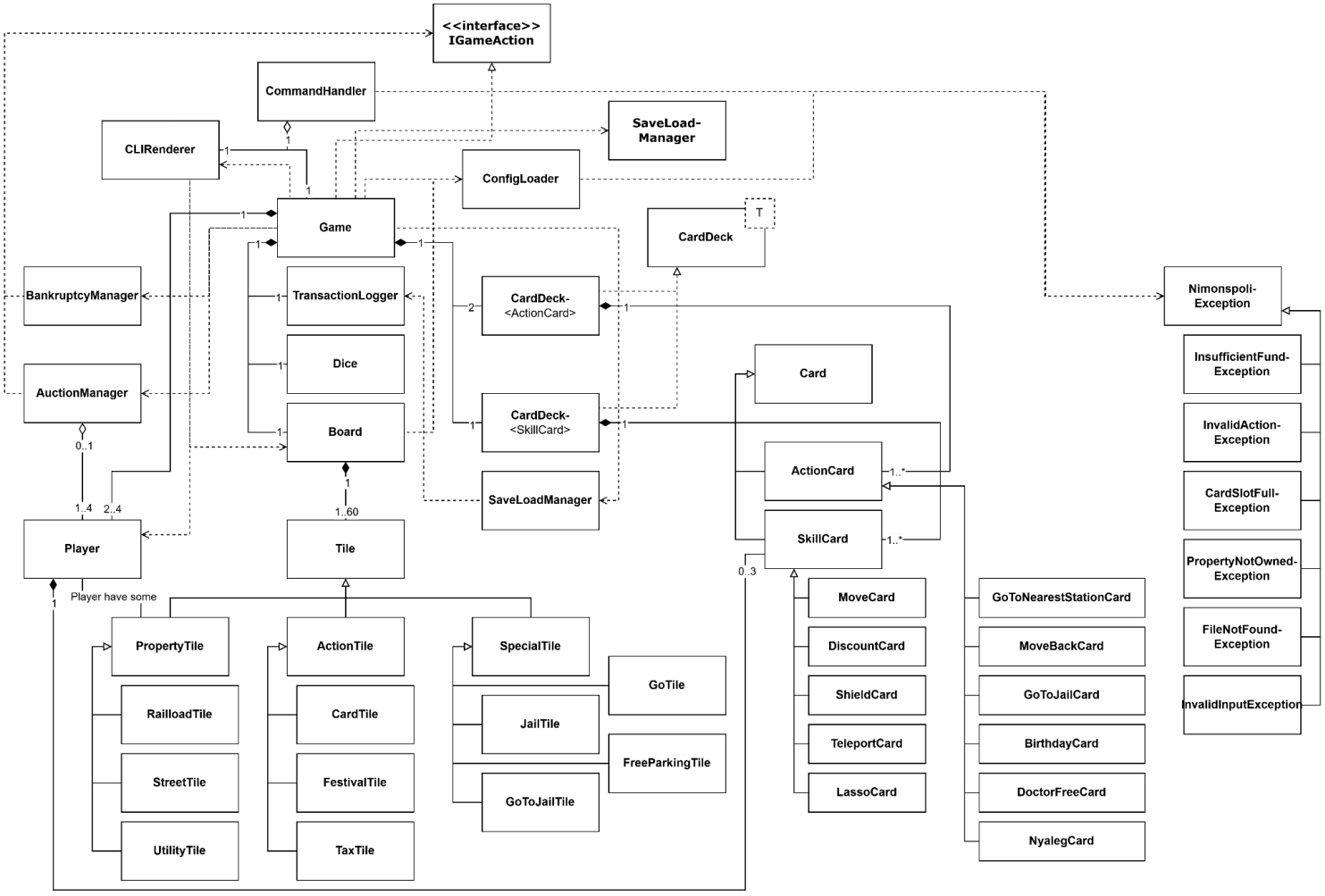
Laporan M2 Tugas Besar 1
IF2010 Pemrograman Berorientasi Objek 2025/2026
Nimonspoli

Kode Kelompok : JAK
Nama Kelompok : JavaAdalahKoentji

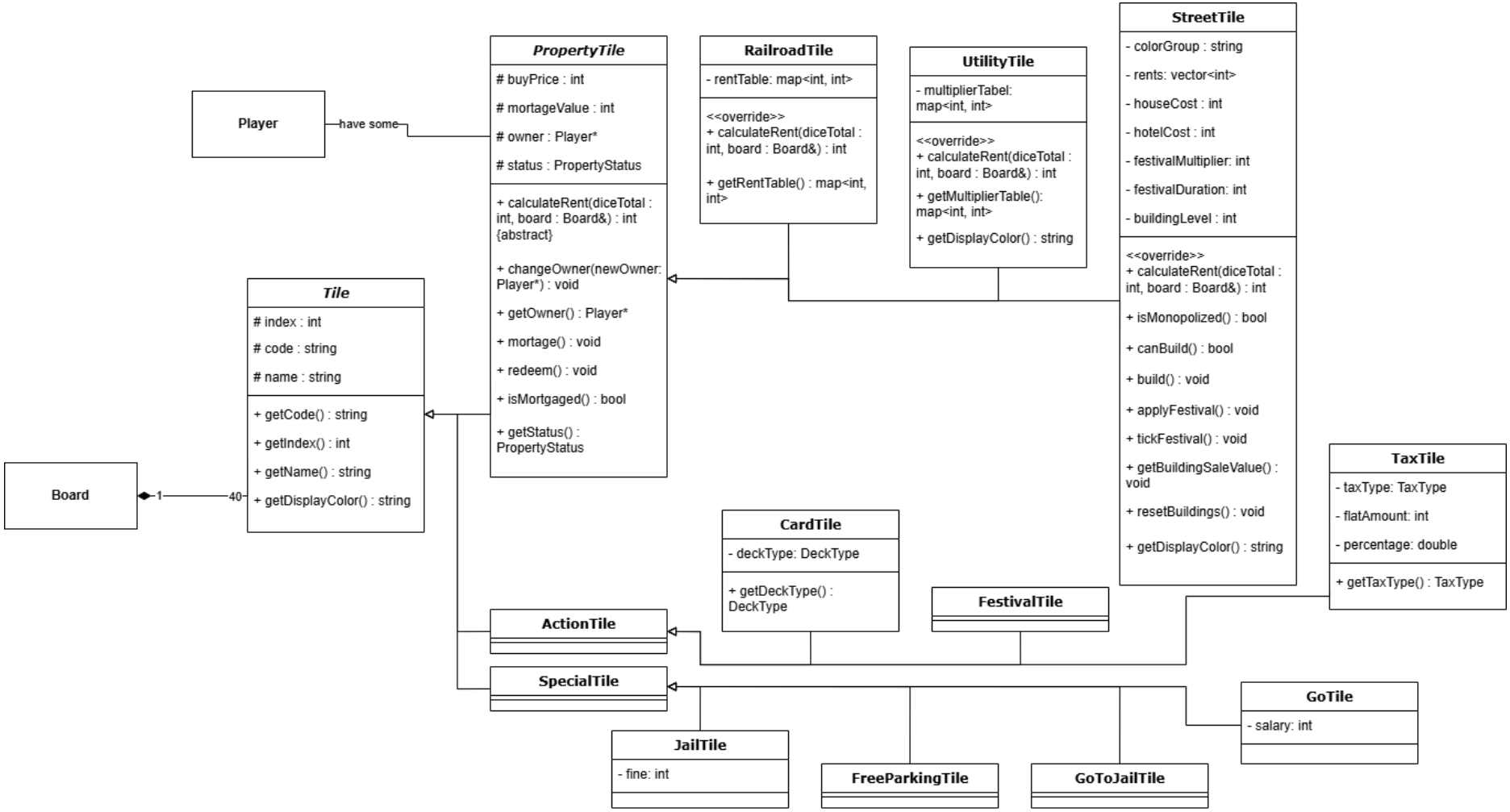
1. 13524128 / Safira Berlianti
2. 13524131 / Amanda Aurellia Salsabilla
3. 13524134 / Salman Faiz Assidqi
4. 13524137 / Reysha Syafitri Mulya Ramadhan
5. 13524138 / Ahmad Rinofaros Muchtar

JAK - JavaAdalahKoentji

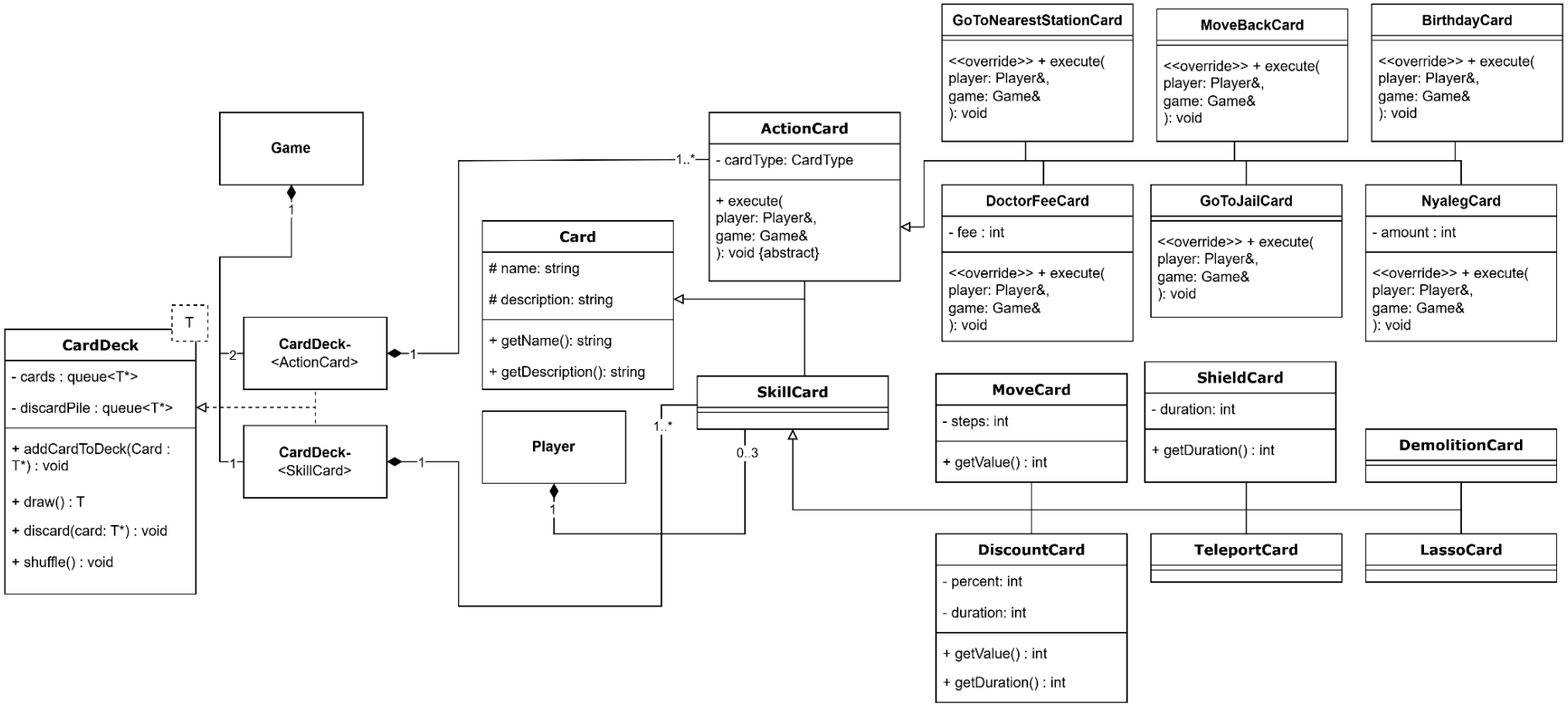
1. Diagram Kelas



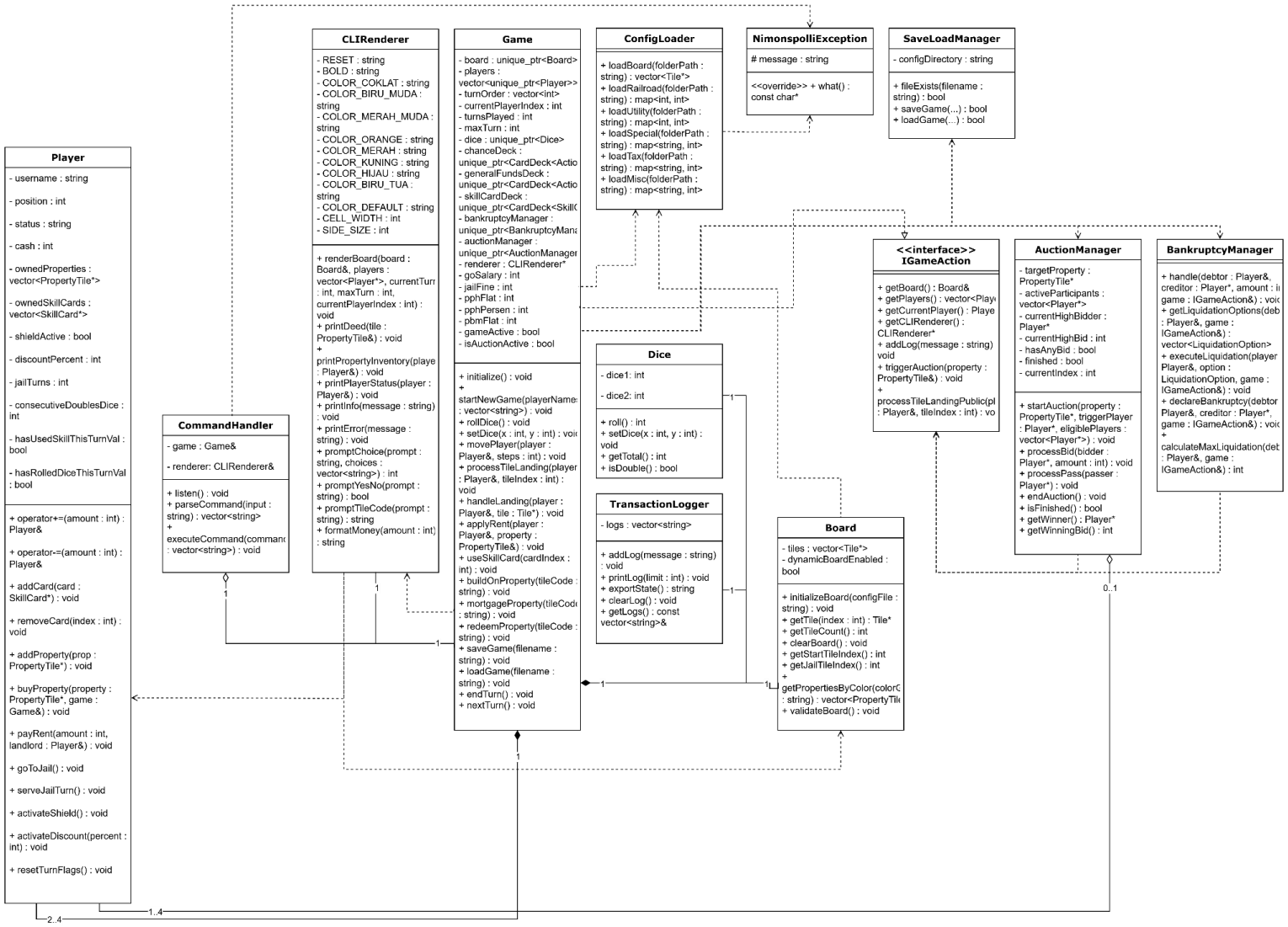
Gambar 1.1 Diagram Kelas Keseluruhan



Gambar 1.2. Diagram Kelas Tile



Gambar 1.3 Diagram Kelas Card



Gambar 1.4 Diagram Kelas Game

2. Deskripsi Kelas

2.1. Kelas Tile

Hierarki Tile merepresentasikan seluruh petak pada papan Nimonspoli. Pada implementasi saat ini, objek Tile tetap menjadi struktur dasar yang disimpan oleh Board dalam vector<Tile*>, sedangkan pemrosesan landing utama dilakukan oleh Game melalui processTileLanding() dan handler seperti handleStreetLanding(), handleRailroadLanding(), handleTaxLanding(), handleFestivalLanding(), dan handleCardLanding(). Dengan demikian, beberapa method onLanded() pada subclass tetap ada sebagai bagian dari desain awal, tetapi tidak selalu menjadi jalur utama eksekusi permainan.

Tile adalah kelas dasar untuk seluruh petak. Kelas ini menyimpan identitas dasar berupa index, code, dan name. Method getDisplayColor() digunakan oleh renderer untuk menentukan tampilan warna petak. Method onLanded() tetap menjadi kontrak umum, tetapi dalam implementasi permainan saat ini Game lebih banyak melakukan dispatch berdasarkan tipe konkret tile.

Atribut:
- index: int → berperan sebagai penanda posisi petak pada papan
- code: string → kode unik petak
- name: string → nama lengkap pada petak

Method:
- getDisplayColor() : string (virtual) → mengembalikan kode warna tampilan
- getCode() : string → getter kode petak
- getIndex() : int → getter indeks petak
- getName() : string → getter nama petak

Relasi:
- Diturunkan oleh PropertyTile, ActionTile, dan SpecialTile
- Diagregasi oleh Board sebagai vector<Tile*>

2.1.1. Kelas PropertyTile

Kelas PropertyTile adalah turunan Tile untuk petak yang dapat memiliki status kepemilikan. Kelas ini menyimpan harga, nilai gadai, pemilik, status properti, serta state efek festival yang dapat berlaku pada semua jenis properti. Perhitungan sewa tetap dibuat polimorfik karena Street, Railroad, dan Utility memiliki formula sewa berbeda.

Atribut:

- buyPrice : int → harga beli properti
- mortgageValue : int → nilai uang yang diterima saat properti digadaikan
- owner : Player* → pointer ke pemilik; nullptr jika belum dimiliki
- status : PropertyStatus → status properti, yaitu BANK, OWNED, atau MORTGAGED

Method:

- calculateRent(diceTotal: int, Board& board) : int (virtual) → menghitung sewa sesuai jenis properti
- mortgage() : void → mengubah status properti menjadi MORTGAGED
- redeem() : void → mengubah status properti menjadi OWNED jika properti memiliki owner
- changeOwner(newOwner: Player*) : void → mengganti pointer owner properti
- getOwner() : Player* → getter pemilik properti
- isMortgaged() : bool → mengecek apakah properti sedang digadaikan
- getPrice() : int → getter harga beli properti
- getMortgageValue() : int → getter nilai gadai properti
- getStatus() : PropertyStatus → getter status properti

Relasi:

- Mewarisi Tile
- Diturunkan oleh StreetTile, RailroadTile, dan UtilityTile
- Berasosiasi dengan Player sebagai pemilik

2.1.1.1. Kelas StreetTile

StreetTile merepresentasikan lahan berwarna yang dapat memiliki bangunan. Kelas ini menyimpan color group, tabel sewa, level bangunan, biaya rumah/hotel, serta state efek festival. Dalam implementasi saat ini,

pembelian, lelang, dan pembayaran sewa street diproses oleh Game::handleStreetLanding(), sedangkan StreetTile menyediakan data dan helper seperti calculateRent(), isMonopolized(), canBuild(), build(), applyFestival(), dan tickFestival().

Atribut:

- colorGroup : string → kelompok warna lahan
- rents : vector<int> → tabel sewa level bangunan 0 sampai 5
- buildingLevel : int → level bangunan; 0 berarti kosong, 1-4 rumah, dan 5 hotel
- festivalMultiplier: int → pengali efek festival
- festivalDuration: int → durasi efek festival dalam giliran
- houseCost : int → biaya membangun satu rumah
- hotelCost : int → biaya membangun hotel

Method:

- calculateRent(diceTotal: int, Board& board) : int (override) → menghitung sewa berdasarkan level bangunan, monopoli, dan festival
- canBuild() : bool → mengecek syarat bangun berdasarkan kepemilikan, monopoli, status gadai, batas level, dan pemerataan bangunan
- build() : void → menaikkan buildingLevel jika canBuild bernilai true
- sell() : void → mengembalikan properti ke BANK, menghapus owner, dan mereset bangunan
- applyFestival(): void → menerapkan efek festival
- tickFestival(): void → mengurangi durasi festival dan reset pengali jika durasi habis
- isMonopolized(): bool → cek apakah properti dalam grup warna yang sama dimonopoli oleh pemiliks
- getDisplayColor() : string (override) → mengembalikan colorGroup untuk pewarnaan papan
- getBuildingSaleValue() : int → menghitung nilai jual bangunan sebesar setengah harga beli
- resetBuildings() : void (override) → mereset buildingLevel serta state festival

Relasi:

- Mewarisi PropertyTile

- Berasosiasi dengan Player sebagai pemilik dan dengan StreetTile lain dalam satu color group

2.1.1.2. Kelas RailroadTile

RailroadTile merepresentasikan stasiun. Kelas ini menyimpan rentTable untuk memetakan jumlah railroad yang dimiliki owner ke nilai sewa. Kepemilikan otomatis dan pembayaran sewa pada alur permainan saat ini ditangani oleh Game::handleRailroadLanding(), sementara RailroadTile tetap menyediakan calculateRent() dan data rentTable.

Atribut:
- rentTable : map<int, int> → pemetaan jumlah railroad yang dimiliki ke biaya sewa

Method:
- calculateRent(diceTotal: int, Board& board) : int (override) → menghitung sewa berdasarkan jumlah railroad milik owner dan multiplier festival
- getRentTable() : const map<int, int>& → getter tabel sewa railroad

Relasi:
- Mewarisi PropertyTile
- Berasosiasi dengan Player sebagai pemilik

2.1.1.3. Kelas UtilityTile

UtilityTile merepresentasikan utilitas seperti PLN dan PAM. Kelas ini menyimpan multiplierTable, yaitu pemetaan jumlah utility milik owner terhadap faktor pengali total dadu. Sama seperti RailroadTile, alur kepemilikan otomatis dan sewa diproses oleh Game::handleUtilityLanding(), sedangkan UtilityTile menyediakan calculateRent(), getMultiplierTable(), dan getDisplayColor().

Atribut:
- multiplierTable : map<int, int> → pemetaan jumlah utility yang dimiliki ke faktor pengali sewa

Method:

- calculateRent(diceTotal: int, Board& board) : int (override) → menghitung sewa dari diceTotal dikali faktor pengali dan multiplier festival
- getDisplayColor() : string (override) → mengembalikan "ABU_ABU" untuk tampilan papan
- getMultiplierTable() : const map<int, int>& → getter tabel multiplier utility

Relasi:

- Mewarisi PropertyTile
- Berasosiasi dengan Player sebagai pemilik

2.1.2. Kelas ActionTile

ActionTile adalah kelas dasar untuk petak aksi non-properti. Pada implementasi saat ini, subclass action tile terutama berfungsi sebagai penanda tipe dan penyimpan konfigurasi, sedangkan efek landing dijalankan oleh Game melalui handler khusus.

Atribut:

- Tidak ada atribut tambahan

Method:

- Tidak ada method khusus yang perlu ditonjolkan

Relasi:

- Mewarisi Tile
- Diturunkan oleh CardTile, FestivalTile, TaxTile

2.1.2.1. Kelas CardTile

CardTile merepresentasikan petak kartu Kesempatan dan Dana Umum. Kelas ini menyimpan deckType yang menentukan deck mana yang dipakai. Saat pemain mendarat, Game::handleCardLanding() membaca deckType, mengambil kartu dari deck yang sesuai, menjalankan execute(), lalu membuang kartu ke discard pile.

Atribut:

- deckType : DeckType → menentukan deck yang digunakan, yaitu CHANCE atau GENERAL_FUNDS

Method:

- getDeckType() : DeckType → geme: Game&) : void (override) → menarik kartu Kesempatan atau Dana Umum melalui Gametter jenis deck

Relasi:

- Mewarisi ActionTile

2.1.2.2. Kelas FestivalTile

FestivalTile merepresentasikan petak festival. Dalam implementasi terbaru, pemilihan properti dan penerapan festival tidak dilakukan di FestivalTile, melainkan di Game::handleFestivalLanding(). Oleh karena itu, FestivalTile cukup dipahami sebagai tipe petak yang memicu handler festival di Game.

Atribut:

- Tidak ada atribut tambahan

Method:

- Tidak ada method khusus yang perlu ditonjolkan

Relasi:

- Mewarisi ActionTile

2.1.2.3. Kelas TaxTile

Kelas TaxTile merepresentasikan petak pajak PPH dan PBM. PPH memberi pilihan membayar flat atau persentase kekayaan, sedangkan PBM langsung memotong nominal flat. Jika shield aktif, pemain dibebaskan dari pajak pada petak tersebut.

Atribut:

- taxType : TaxType → jenis pajak, yaitu PPH atau PBM
- flatAmount : int → nominal pajak flat sesuai konfigurasi
- percentage : double → persentase kekayaan untuk opsi PPH

Method:

- getTaxType(): TaxType → getter atribut taxType

Relasi:

- Mewarisi ActionTile

2.1.3. Kelas SpecialTile

SpecialTile adalah kelas dasar untuk petak khusus seperti GO, Penjara, Pergi ke Penjara, dan Bebas Parkir. Semua efek khusus saat pemain mendarat pada implementasi saat ini juga ditangani oleh Game.

Atribut:

- Tidak ada atribut tambahan

Method:

- Tidak ada method khusus selain pewarisan dari Tile

Relasi:

- Mewarisi Tile
- Diturunkan oleh GoTile, JailTile, GoToJailTile, dan FreeParkingTile

2.1.3.1. Kelas GoTile

Kelas GoTile merepresentasikan petak mulai. Pemain menerima salary baik ketika berhenti tepat di GO maupun ketika melewati GO melalui pemanggilan onPassed() dari Game.

Atribut:

- salary : int → besaran gaji sesuai konfigurasi

Method:
- Tidak ada method khusus yang perlu ditonjolkan

Relasi:
- Mewarisi SpecialTile

2.1.3.2. Kelas JailTile

Kelas JailTile merepresentasikan petak penjara. Saat pemain mendarat, kelas ini membedakan apakah pemain sedang berstatus JAILED atau hanya mampir.

Atribut:
- fine : int → denda keluar penjara sesuai konfigurasi

Method:
- Tidak ada method khusus yang perlu ditonjolkan

Relasi:
- Mewarisi SpecialTile

2.1.3.3. Kelas GoToJailTile

Kelas GoToJailTile merepresentasikan petak Pergi ke Penjara. Pemain yang mendarat dikirim ke JailTile melalui Game, kecuali sedang memiliki efek shield yang aktif.

Atribut:
- Tidak ada atribut tambahan

Method:
- Tidak ada method khusus yang perlu ditonjolkan

Relasi:

- Mewarisi SpecialTile

2.1.3.4. Kelas FreeParkingTile

Kelas FreeParkingTile merepresentasikan petak Bebas Parkir. Petak ini tidak memberikan efek finansial maupun perubahan status ketika diinjak.

Atribut:

- Tidak ada atribut tambahan

Method:

- Tidak ada method khusus yang perlu ditonjolkan

Relasi:

- Mewarisi SpecialTile

2.1.4. Enum Pendukung

Hierarki Tile menggunakan enum sebagai tipe pendukung agar status dan kategori objek lebih aman dan konsisten dibandingkan string bebas. PropertyStatus digunakan untuk status BANK, OWNED, dan MORTGAGED. DeckType digunakan oleh CardTile untuk membedakan deck CHANCE dan GENERAL_FUNDS. TaxType digunakan oleh TaxTile untuk membedakan PPH dan PBM. PropertyType dan TileType dapat digunakan sebagai penanda kategori class jika dibutuhkan oleh header atau diagram.

2.2. Kelas Dice

Kelas Dice bertanggung jawab penuh atas mekanisme pengacakan dan penentuan langkah pemain menggunakan dua buah dadu bersisi 6 (2d6). Kelas ini memiliki peran penting untuk menentukan pergerakan pemain. Kelas ini berelasi erat dengan Game dalam alur giliran pemain.

Atribut:

- dice1 : int → menyimpan nilai dadu pertama.
- dice2 : int → menyimpan nilai dadu kedua.

Method:

- roll() : int → mengacak nilai dadu secara random dan mengembalikan totalnya.
- setDice(x: int, y: int) : int → mengatur hasil lemparan dadu secara manual sesuai perintah ATUR_DADU X Y.
- getDice1() : int → ambil nilai dadu 1
- getDice2() : int → ambil nilai dadu 2
- getTotal() : int → ambil jumlah nilai dadu 1 ditambah dadu 2
- isDouble() : bool → mengecek apakah nilai kedua dadu sama.

Relasi:

- Dimiliki oleh Game

2.3. Kelas Card

Kelas Card merupakan kelas abstrak fundamental (abstract class) yang menjadi fondasi bagi semua kartu di Nimonspoli, baik itu kartu aksi maupun kartu kemampuan. Kelas ini memungkinkan implementasi polimorfisme saat kartu disimpan dalam tumpukan (deck) atau di tangan pemain.

Atribut:

- name : string → nama dari kartu.
- description : string → deskripsi efek kartu.

Method:

- getName() : string → getter nama kartu.
- getDescription() → getter deskripsi kartu.

Relasi:

- Diturunkan oleh ActionCard dan SkillCard

2.3.1. Kelas ActionCard

Kelas turunan dari Card yang merepresentasikan kartu dari tumpukan Kesempatan (Chance) dan Dana Umum (Community Chest). Tanggung jawabnya adalah memberikan efek instan kepada pemain saat mereka mendarat di petak yang sesuai.

Atribut:
- cardType : CardType → jenis kartu: CHANCE atau COMMUNITY_CHEST

Method:
- execute(player: Player&, game: Game&) : void (override) → menjalankan instruksi instan tersebut, lalu kartu akan dikembalikan dan di-reshuffle ke dalam tumpukannya.

Relasi:
- Mewarisi Card
- Diturunkan menjadi GoToNearestStationCard, MoveBackCard, GoToJailCard, BirthdayCard, DoctorFeeCard, NyalegCard
- Dikelola oleh CardDeck<ActionCard>

2.3.1.1. Kelas GoToNearestStationCard

Kelas turunan dari ActionCard yang merepresentasikan kartu Kesempatan dengan instruksi memindahkan pemain ke stasiun terdekat dari posisinya saat ini.

Atribut:
- Tidak ada atribut tambahan

Method:

- execute(player: Player&, game: Game&) : void (override) → mencari stasiun terdekat dari posisi pemain saat ini dan memindahkan bidaknya ke sana, lalu memicu onLanded() di petak stasiun tersebut.

Relasi:
- Mewarisi ActionCard

2.3.1.2. Kelas MoveBackCard

Kelas turunan dari ActionCard yang merepresentasikan kartu Kesempatan dengan instruksi memundurkan pemain sebanyak 3 petak dari posisinya saat ini.

Atribut:
- Tidak ada atribut tambahan

Method:
- execute(player: Player&, game: Game&) : void (override) → memundurkan posisi pemain sebanyak 3 petak dan memicu onLanded() di petak tujuan.

Relasi:
- Mewarisi ActionCard

2.3.1.3. Kelas GoToJailCard

Kelas turunan dari ActionCard yang merepresentasikan kartu Kesempatan dengan instruksi mengirim pemain langsung ke penjara.

Atribut:
- Tidak ada atribut tambahan

Method:

- execute(player: Player&, game: Game&) : void (override) → memindahkan bidak pemain ke JailTile dan mengubah statusnya menjadi JAILED.

Relasi:
- Mewarisi ActionCard

2.3.1.4. Kelas BirthdayCard

Kelas turunan dari ActionCard yang merepresentasikan kartu Dana Umum dengan instruksi pemain menerima uang dari seluruh pemain lain sebagai hadiah ulang tahun.

Atribut:
- amount : int → jumlah uang yang diterima dari tiap pemain (M100)

Method:
- execute(player: Player&, game: Game&) : void (override) → memotong saldo setiap pemain lain sebesar amount dan menambahkan totalnya ke saldo pemain yang memegang kartu ini.

Relasi:
- Mewarisi ActionCard

2.3.1.5. Kelas DoctorFeeCard

Kelas turunan dari ActionCard yang merepresentasikan kartu Dana Umum dengan instruksi pemain membayar biaya dokter ke Bank.

Atribut:
- fee : int → jumlah yang harus dibayar (M700)

Method:

- execute(player: Player&, game: Game&) : void (override) → memotong saldo pemain sebesar fee dan membayarkannya ke Bank. Jika tidak mampu bayar, kontrol diserahkan ke BankruptcyManager.

Relasi:
- Mewarisi ActionCard

2.3.1.6. Kelas NyalegCard

Kelas turunan dari ActionCard yang merepresentasikan kartu Dana Umum dengan instruksi pemain membayar sejumlah uang ke setiap pemain lain.

Atribut:
- amount : int → jumlah yang harus dibayar ke tiap pemain (M200)

Method:
- execute(player: Player&, game: Game&) : void (override) → memotong saldo pemain sebesar amount untuk setiap pemain lain dan mendistribusikannya. Jika tidak mampu bayar, kontrol diserahkan ke BankruptcyManager.

Relasi:
- Mewarisi ActionCard

2.3.2. Kelas SkillCard

Kelas turunan dari Card yang merepresentasikan Kartu Kemampuan Spesial. Tanggung jawabnya adalah memberikan keuntungan taktis yang dapat disimpan di tangan pemain dan diaktifkan secara manual.

Method:
- use(player: Player&, game: Game&) : void (pure virtual) → mengaktifkan efek kartu

Relasi:

- Mewarisi Card
- Diturunkan menjadi MoveCard, DiscountCard, ShieldCard, TeleportCard, LassoCard, DemolitionCard
- Dimiliki oleh Player (0–3 kartu)
- Dikelola oleh CardDeck<SkillCard>

2.3.2.1. Kelas MoveCard

Kelas ini merepresentasikan kartu kemampuan yang memungkinkan pemain untuk bergerak maju sekian petak.

Atribut:
- steps : int → jumlah langkah pemain dapat maju

Method:
- use(player: Player&, game: Game&) : void (override) → membuat pemain maju sebanyak steps

2.3.2.2. Kelas DiscountCard

Kelas ini merepresentasikan kartu kemampuan yang dapat memberikan diskon dengan persentase acak kepada pemain saat kartu didapatkan.

Atribut:
- percent : int → persentase diskon yang didapatkan
- duration : int → sisa durasi efek kartu

Method:
- use(player: Player&, game: Game&) : void (override) → membuat pemain mengaktifkan diskon

2.3.2.3. Kelas ShieldCard

Kelas ini merepresentasikan kartu kemampuan yang dapat melindungi pemain dari tagihan sewa maupun sanksi apapun yang merugikan selama 1 giliran.

Atribut:

- duration : int → sisa durasi efek kartu

Method:

- use(player: Player&, game: Game&) : void (override) → membuat pemain kebal dari sewa maupun sanksi

2.3.2.4. Kelas TeleportCard

Kelas ini merepresentasikan kartu kemampuan yang memberikan pemain kebebasan untuk berpindah ke petak manapun di atas papan permainan.

Method:

- use(player: Player&, game: Game&) : void (override) → membuat pemain berpindah ke petak yang dipilih

2.3.2.5. Kelas LassoCard

Kelas ini merepresentasikan kartu kemampuan untuk menarik satu pemain lawan yang berada di depan posisi pemain saat ini ke petak tempat pemain tersebut berada.

Method:

- use(player: Player&, game: Game&) : void (override) → menarik pemain lawan

2.3.2.6. Kelas DemolitionCard

Kelas ini merepresentasikan kartu kemampuan yang dapat menghancurkan satu properti milik pemain lawan.

Method:

- use(player: Player&, game: Game&) : void (override) → menghancurkan satu properti milik lawan

2.4. Generic Class CardDeck<T>

Kelas generik (CardDeck<T>) yang bertanggung jawab mengelola tumpukan dari berbagai jenis kartu permainan secara fleksibel.

Atribut:

- cards : queue<T*> → antrean kartu yang siap ditarik.
- discardPile : vector<T*> → tumpukan pembuangan untuk kartu yang telah terpakai.

Method:

- draw() : T* → mengambil kartu teratas. Jika cards habis, method ini otomatis mengocok ulang (reshuffle) semua kartu dari discardPile untuk membentuk cards baru.
- discard(card: T*) : void → membuang kartu yang telah dipakai ke discardPile.
- getStateInfo() : string → mengekstrak jumlah dan jenis kartu untuk keperluan format State Deck saat penyimpanan.

Relasi:

- Digunakan oleh CardTile untuk ActionCard
- Digunakan oleh Game untuk SkillCard

2.5. Kelas Player

Kelas ini merepresentasikan pemain dalam permainan Nimonspoli. Kelas ini bertanggung jawab mengelola logika pemain, seperti status, posisi, efek aktif, kartu, dan properti dari seorang pemain.

Atribut:

- username : string → nama pemain
- position : int → indeks lokasi pemain pada papan
- status : string → status dari pemain yang bernilai ACTIVE, BANKRUPT, atau JAILED
- cash : int → saldo uang yang dimiliki pemain
- ownedProperties : vector<PropertyTile*> → daftar properti yang dimiliki pemain
- ownedSkillCards : vector<SkillCard*> → daftar kartu skill yang dimiliki pemain
- shieldActive : boolean → status perlindungan dari serangan kartu lawan
- discountPercent : int → persentase potongan harga sewa yang sedang dimiliki
- jailTurns : int → counter durasi pemain di penjara penjara

- consecutiveDoublesDice : int → counter jumlah pemain mendapatkan dadu kembar berturut-turut
- hasUsedSkillThisTurn : boolean → menandai apakah pemain sudah menggunakan skill pada suatu giliran
- gasRolledDice : boolean → menandai apakah pemain sudah mengocok dadu

Method:

- operator+=(int amount) : Player& → menambah saldo pemain
- operator-=(int amount): Player& → mengurangi saldo pemain
- operator>(const Player& o) : boolean → membandingkan kekayaan pemain
- operator<(const Player& o) : boolean → membandingkan kekayaan pemain
- getTotalWealth() : int → menghitung total seluruh kekayaan pemain (Jumlah Kas + Harga Beli Properti + Harga Bangunan)
- getTotalPropertyPrice() : int → menghitung total harga beli dari semua petak properti yang dimiliki pemain
- getTotalBuildingPrice() : int → menghitung total nilai investasi bangunan (rumah/hotel) yang telah didirikan di semua properti pemain
- getMaxLiquidation() : int → menghitung jumlah uang tunai maksimal yang bisa didapatkan pemain jika seluruh asetnya dijual atau digadaikan saat ini
- addCard(SkillCard *card) : void → menambahkan objek kartu skill baru ke dalam daftar inventaris kartu yang dimiliki pemain
- removeCard(int index) : void → menghapus kartu skill dari inventaris pemain berdasarkan indeks tertentu (biasanya dipanggil setelah kartu digunakan)
- addProperty(PropertyTile *prop) : void → menambahkan pointer properti ke dalam daftar koleksi hak milik pemain
- buyProperty(PropertyTile *property, Game &game) : void → melakukan transaksi pembelian properti (mengurangi saldo pemain, mengubah status pemilik, dan mencatat log transaksi)
- payRent(int amount, Player &landlord) : void → melakukan transfer uang dari pemain ke pemilik tanah (landlord) sebagai pembayaran sewa
- calculatePayableRent(int amount) : int→ menghitung berapa banyak dari total uang sewa yang sanggup dibayar pemain dengan kas yang tersedia sebelum harus melakukan likuidasi
- goToJail(int jailTileIndex) : void → memindahkan posisi pemain ke petak penjara dan mengubah status pemain menjadi dipenjara
- serveJailTurn() : void → mengurangi sisa durasi masa tahanan pemain di dalam penjara sebanyak satu giliran
- releaseFromJail() : void → mengembalikan status pemain menjadi aktif (bebas) sehingga bisa bergerak kembali di papan

- resetTurnFlags() : void → mengatur ulang semua penanda aksi (seperti status sudah lempar dadu atau sudah pakai skill) agar pemain siap memulai giliran baru
- hasRolledDiceThisTurn() : void → mengecek apakah pemain sudah melakukan lempar dadu pada giliran yang sedang berjalan
- setRolledDiceThisTurn(bool status) : void → mengubah status penanda lempar dadu (mencegah pemain melempar dadu dua kali dalam satu giliran)
- handleBankruptcy(Player &creditor, int amountOwed) : void → menangani proses penyerahan aset atau pernyataan kalah ketika utang pemain tidak tertutup oleh nilai likuidasi
- liquidateAssets(int targetAmount) : void → memproses penjualan atau penggadaian aset secara otomatis atau manual hingga mencapai jumlah uang yang ditargetkan untuk membayar utang
- removeProperty(PropertyTile* prop) : void → menghapus satu properti spesifik dari daftar kepemilikan pemain
- clearProperties() : void → menghapus seluruh daftar kepemilikan properti pemain (biasanya saat keluar dari game atau kalah)
- clearSkillCards() : void → menghapus seluruh kartu skill yang sedang dipegang oleh pemain
- releaseAllProperties() : vector<PropertyTile*> → mengosongkan daftar kepemilikan pemain dan mengembalikan kumpulan pointer properti tersebut (digunakan untuk menyerahkan aset ke Bank atau pemain lain saat bangkrut)

Relasi:

- Player menyimpan vector of Property.
- Player memiliki vector of SkillCard.
- Player berinteraksi dengan Tile melalui metode onLanded(Player&, Game&).
- Player bertindak sebagai participant dalam sebuah proses lelang (AuctionManager).
- Player akan melempar exception dari NimonspoliException jika melakukan aksi ilegal, seperti membeli properti namun uangnya tidak cukup.

2.6. Kelas AuctionManager

Kelas ini bertanggung jawab untuk mengelola proses lelang properti, termasuk penawaran, pass, dan penentuan pemenang.

Atribut:

- targetProperty : Property* → pointer ke properti yang sedang dilelang

- activeParticipants : vector<Player*> → daftar pemain yang ikut serta dalam putaran lelang
- currentHighBidder : Player* → pointer ke pemain yang saat ini memiliki tawaran tertinggi
- currentHighBid : int → nilai tawaran tertinggi saat ini
- hasAnyBid : boolean → menandai apakah sudah ada minimal satu tawaran yang masuk dalam lelang tersebut
- consecutivePasses : int → jumlah pass berturut-turut
- currentIndex : int → indeks pemain yang sedang menawar
- finished : boolean → menandai apakah lelang sudah selesai
- gameAction : IGameAction* → interface untuk melakukan komunikasi balik ke logika permainan utama

Method:

- advanceToNextParticipant() : void → memindahkan giliran ke pemain berikutnya
- endAuction() : void → dipanggil ketika syarat lelang selesai terpenuhi. Method ini menentukan apakah ada pemenang atau tidak, melakukan log hasil akhir, dan memanggil gameAction->finalizeAuction untuk mengeksekusi perpindahan kepemilikan properti.
- logEvent(const std::string& action, const std::string& detail) : void → mengirimkan catatan aktivitas lelang ke sistem log melalui objek gameAction
- startAuction(PropertyTile* property, Player* triggerPlayer, const vector<Player*>& eligiblePlayers) : void → memulai proses lelang dengan properti tertentu dan daftar pemain yang ikut
- processBid(Player* bidder, int amount) : void → memproses tawaran dari pemain dan mengecek apakah amount lebih besar dari currentHighBid
- processPass(Player* passer) : void → menangani pemain yang memilih untuk tidak menaikkan tawaran
- isFinished() : bool → mengecek apakah lelang sudah selesai
- getWinner() : Player* → mengembalikan pemenang lelang (currentHighBidder) jika lelang sudah selesai
- getWinningBid() : int → mengembalikan nilai nominal harga akhir yang dimenangkan
- getTargetProperty() : PropertyTile* → Mengembalikan pointer ke properti yang dilelang

Relasi:

- AuctionManager menggunakan PropertyTile sebagai target properti yang sedang dilelang.
- AuctionManager berinteraksi dengan Player sebagai peserta lelang (activeParticipants) dan penawar tertinggi (currentHighBidder).

- AuctionManager bergantung pada IGameAction untuk mencatat log kejadian lelang dan memfinalisasi hasil lelang ke sistem utama.
- AuctionManager melempar InvalidActionException jika pemain melakukan aksi di luar giliran atau saat lelang sudah berakhir.
- AuctionManager melempar InsufficientFundsException jika pemain memberikan tawaran (*bid*) yang melebihi saldo kas yang dimiliki.

2.7. Kelas BankruptcyManager

Kelas ini menangani kondisi ketika pemain tidak mampu membayar kewajibannya. Pertama kelas ini menghitung apakah likuidasi aset bisa menutup kewajiban. Jika bisa, pemain diarahkan ke panel likuidasi. Jika tidak, pemain langsung dinyatakan bangkrut dan seluruh asetnya dialihkan ke kreditor atau dikembalikan ke Bank.

Method:

- getSellValue(const PropertyTile& prop) : int → menghitung nilai jual properti, termasuk nilai bangunan
- isMortgageable(const PropertyTile& prop, const IGameAction& game) : boolean → mengecek apakah suatu properti bisa digadaikan
- calculateMaxLiquidation(const Player& debtor, const IGameAction& game) : int → menghitung total dana maksimal yang bisa didapatkan pemain dari sisa uang dan seluruh aset yang bisa dijual/digadaikan.
- canCoverWithLiquidation(const Player& debtor, int obligation, const IGameAction& game) : boolean → mengecek apakah total nilai likuidasi pemain mampu menutupi jumlah kewajiban
- getLiquidationOptions(player: Player*, board: Board&) : vector<LiquidationOption> → mengembalikan daftar opsi jual/gadai yang tersedia
- executeLiquidation(player: Player*, option: LiquidationOption, board: Board&) : void → menjalankan satu aksi likuidasi yang dipilih pemain
- declareBankruptcy(debtor: Player*, creditor: Player*, game: Game&) : void → mendeklarasikan bangkrut dan mengalihkan semua aset

- handle(Player& debtor, Player* creditor, int obligation, IGameAction& game, function<int(const vector<LiquidationOption>&)> chooseOption) → titik masuk utama untuk menangani kewajiban pembayaran. Jika uang tidak cukup, metode ini akan menjalankan siklus likuidasi hingga utang lunas atau menyatakan bangkrut jika total aset tidak mencukupi.

Relasi:

- BankruptcyManager diinisialisasi dan dipanggil oleh kelas utama yang mengimplementasikan IGameAction saat terjadi transaksi yang gagal karena saldo tidak cukup.
- BankruptcyManager berinteraksi intensif dengan Player untuk memanipulasi kas, status pemain (BANKRUPT), dan daftar properti.
- BankruptcyManager memicu AuctionManager (melalui game.triggerAuction) jika pemain bangkrut kepada Bank, agar aset yang dilepas bisa dilelang kembali ke pemain lain.
- BankruptcyManager menggunakan TransactionLogger untuk mencatat setiap aksi jual, gadai, atau pernyataan bangkrut ke dalam log permainan.
- BankruptcyManager bergantung pada PropertyTile dan turunannya (StreetTile) untuk menghitung nilai aset dan memproses perubahan status properti.

2.8. Kelas TransactionLogger

Kelas ini berfungsi mencatat setiap kejadian signifikan secara otomatis di latar belakang ke dalam log terstruktur tanpa memerlukan perintah khusus.

Atribut:

- logs : vector<string> → koleksi rekaman transaksi (menggunakan STL Vector).

Method:

- addLog(turn: int, username: string, actionType: string, detail: string) : void → menambahkan entri log baru yang memuat informasi wajib: nomor turn, username, jenis aksi (misal: DADU, BELI, SEWA), dan detail.
- getLogs() : vector<string> → mendapatkan log yang sudah tercatat dalam vector
- printLog(limit: int) : void → mencetak semua log ke layar atau membatasinya sesuai argumen jumlah baris tertentu, sesuai perintah CETAK_LOG.

- clearLog() : void → menghapus semua log
- exportState() : string → mengeluarkan full log dari memori ke dalam format teks untuk disimpan ke dalam save file.

Relasi:

- Dimiliki oleh Game
- Diakses oleh SaveLoadManager saat proses save

2.9. Kelas Board

Kelas Board merepresentasikan struktur papan permainan. Tanggung jawabnya adalah menyimpan dan memanajemen seluruh urutan Tile.

Atribut:

- tiles : vector<Tile*> → koleksi dinamis yang berisi 40 petak pada papan, atau antara 20 hingga 60 petak jika fitur Papan Dinamis diaktifkan.

Method:

- initializeBoard(configFile: string) : void → menyusun petak berdasarkan property.txt dan konfigurasi papan. Wajib memastikan ada tepat 1 Petak Mulai (GO) dan 1 Petak Penjara.
- getTile(index: int) : Tile* → mengembalikan referensi ke suatu petak.
- printBoard(players: vector<Player*>) : void → menangani visualisasi papan sesuai perintah CETAK_PAPAN dengan menampilkan warna kategori properti, posisi bidak, serta legenda kepemilikan.

Relasi:

- Dimiliki oleh Game
- Mengagregasi Tile*

2.10. Kelas Game

Kelas ini merupakan kelas orkestrator utama yang menghubungkan semua komponen dalam game.

Atribut:

- board : Board → objek papan permainan yang berisi semua petak beserta properti di dalamnya
- players : vector<Player*> → daftar pointer ke semua pemain yang sedang bermain
- turnOrder : vector<int> → urutan indeks pemain dalam giliran
- currentTurnIndex : int → indeks dalam turnOrder yang menunjukkan pemain mana yang sedang mendapat giliran
- currentTurn : int → nomor giliran keseluruhan (turn keberapa sejak permainan dimulai)
- chanceDeck : CardDeck<ActionCard> → tumpukan kartu kesempatan
- communityDeck : CardDeck<ActionCard> → tumpukan kartu dana umum
- skillDeck : CardDeck<SkillCard> → tumpukan kartu skill
- dice : Dice → objek dadu untuk menentukan jumlah langkah pemain
- logger : TransactionLogger → pencatat semua aksi penting dalam permainan
- gameActive : bool → menandakan apakah permainan masih berlangsung atau sudah selesai
- winner : Player* → pointer ke pemenang permainan, false jika permainan masih berjalan

Method:

- startNewGame(vector<string> playerNames, int startingCash) : void → menginisialisasi permainan baru dengan membuat objek Player untuk setiap nama, mengatur starting cash, mengacak urutan giliran, mengocok semua deck, mereset posisi semua pemain ke GO, dan memulai turn pertama
- runTurn(Player& player) : void → menjalankan satu giliran untuk pemain tertentu
- nextTurn() : void → berpindah ke giliran pemain berikutnya
- handleLanding(Player& player, Tile* tile) : void → memproses efek ketika pemain mendarat di suatu petak
- handleJail(Player& player) : void → menangani logika pemain yang sedang berada di penjara
- handleAuction(Property* property) : void → menanganani logika pelelangan
- handleBankruptcy(Player& player, int amountOwed) : void → menangani situasi ketika pemain tidak mampu membayar kewajiban (sewa, pajak, denda, atau tawaran lelang)
- handleLiquidation(Player& player, int targetAmount) : int → membantu handleBankruptcy dengan menjual aset pemain hingga mencapai targetAmount; mengembalikan total dana yang berhasil dikumpulkan
- updateEffects() : void → dipanggil setiap akhir giliran untuk mengurangi durasi efek sementara

- checkGameEnd() : bool → mengecek apakah permainan berakhir
- processPlayerInput(string input) : void → memproses input dari pemain selama giliran berlangsung
- getCurrentPlayer() : Player* → mengembalikan pointer ke pemain yang sedang mendapat giliran
- addLog(string actionType, string detail) : void → menambahkan entri ke logger dengan turn dan username pemain saat ini

Relasi:

- Game memiliki satu objek Board. Tanpa Board, Game tidak memiliki tempat untuk menggerakkan pemain.
- Game memiliki objek Dice untuk menentukan langkah pemain setiap kali metode runTurn dijalankan.
- Game memiliki tiga tumpukan kartu (CardDeck<T>), yaitu chanceDeck, communityDeck, dan skillDeck.
- Setiap kali ada kejadian di Game, ia akan memerintahkan logger (TransactionLogger) untuk mencatat.
- Game menyimpan daftar seluruh pemain (vector<Player*>) dan mengelola giliran mereka melalui currentTurnIndex.
- Saat awal dijalankan (startNewGame), Game memanggil ConfigLoader untuk mengisi data awal papan dan pemain.
- Game mengirimkan seluruh state dirinya (pemain, properti, deck) ke SaveLoadManager untuk disimpan ke dalam file txt.
- Game menugaskan BankruptcyManager untuk menangani kebangkrutan dan AuctionManager untuk menangani lelang ketika kondisi tertentu terpenuhi.
- Game menerima perintah yang sudah diparse oleh CommandHandler untuk kemudian dieksekusi logikanya.

2.11. Kelas SaveLoadManager

Sebagai Data Access Layer, kelas ini murni bertugas menangani pembacaan dan penulisan status (state) permainan ke file bereksistensi .txt.

Atribut:

- Tidak memiliki atribut persisten, beroperasi murni dari argumen yang diberikan.

Method:

- savePlayerState() : void → khusus untuk menyimpan state yang dimiliki Player sesuai format
- savePropertyState() : void → khusus untuk menyimpan state yang dimiliki Property sesuai format
- saveDeckState() : void → khusus untuk menyimpan state yang dimiliki Deck sesuai format
- saveLogState() : void → khusus untuk menyimpan state yang dimiliki Log sesuai format

- loadPlayerState() : void → khusus untuk ngeload konfigurasi Player sesuai format
- loadDeckState() : vector<string> → khusus untuk ngeload kartu dari deck sesuai format
- loadLogState() : void → khusus untuk ngeload konfigurasi Log sesuai format
- createSkillCard(type: string, value: int, duration: int) → buat SkillCard dari nama jenis dan nilai yang ada di file konfigurasi
- saveGame(game: Game&, filename: string) : void → diakses melalui perintah SIMPAN. Hanya bisa dieksekusi di awal giliran sebelum aksi apapun. Menuliskan urutan State Pemain, State Properti, State Deck, dan State Log sesuai format yang diwajibkan.
- loadGame(filename: string) : void → diakses melalui perintah MUAT. Hanya bisa dilakukan saat pertama kali program berjalan.
- fileExists(filename: string) : bool → cek apakah file sudah ada atau belum

Relasi:
- Digunakan oleh Game
- Mengakses TransactionLogger untuk menyimpan log

2.12. Kelas NimonspoliException

Kelas Exception Handling khusus yang berfungsi menangani berbagai galat dan kondisi tak valid agar program tidak berhenti tiba-tiba.

Atribut:
- message : string → deskripsi detail kesalahan.

Method:
- what() : const char* (override) → mengembalikan pesan kesalahan.

Relasi:
- Dilempar oleh komponen di Game Logic Layer
- Ditangkap oleh CommandHandler

2.12.1. Kelas InsufficientFundsException

Exception ini dilempar ketika seorang pemain mencoba melakukan transaksi (membeli properti, membangun rumah, membayar denda, atau menawar lelang) namun saldo cash yang dimilikinya tidak mencukupi.

2.12.2. Kelas InvalidActionException

Exception ini dilempar ketika pemain mencoba melakukan aksi yang melanggar logika atau aturan permainan.

2.12.3. Kelas CardSlotFullException

Exception ini dilempar ketika pemain mendapatkan kartu baru namun jumlah kartu yang dimilikinya sudah mencapai batas maksimal, yaitu 3 kartu.

2.12.4. Kelas PropertyNotOwnedException

Exception ini dilempar ketika ada operasi yang dilakukan pada properti yang statusnya bukan milik pemain yang bersangkutan.

2.12.5. Kelas FileNotFoundException

Exception ini dilempar ketika sistem gagal menemukan file .txt yang diperlukan.

2.12.6. Kelas InvalidInputException

Exception ini dilempar ketika input teks dari pengguna tidak sesuai dengan format yang dikenali oleh sistem.

2.13. Kelas ConfigLoader

Kelas ini berada pada Data Access Layer dan bertanggung jawab penuh untuk membaca, mem-parsing, dan memuat semua pengaturan awal dari berkas-berkas .txt (seperti property.txt, railroad.txt, tax.txt, dll) ke dalam memori sebelum permainan dimulai. Dengan memisahkan tanggung jawab ini ke kelas ConfigLoader, kelas Game dan Board tidak perlu tahu bagaimana cara membaca file atau melakukan parsing teks, melainkan cukup menerima data yang sudah matang dan terstruktur.

Atribut:

- Tidak ada atribut tersendiri

Method:

- loadBoard() : vector<Tile> → ngeload papan yang terdiri dari sekumpulan (vector of) Tile
- loadTax() : map<string, int> → membaca tax.txt untuk mengambil nilai persentase dan jumlah flat dari PPH serta PBM.
- loadSpecial() : map<string, int> membaca gaji dari petak GO dan denda keluar penjara dari special.txt
- loadMisc() : map<string, int> membaca misc.txt untuk batas max turn dan saldo awal.
- loadRailroad() : map<int, int> → membaca railroad.txt untuk mendapatkan pemetaan jumlah stasiun terhadap harga sewa.
- loadUtility() : map<int, int> → membaca utility.txt untuk faktor pengali dadu.

Relasi:

- Merupakan utilitas yang dipanggil oleh kelas Game dan Board tepat pada fase inisialisasi awal (perintah New Game).
- Dapat melempar NimonspoliException jika file konfigurasi tidak ditemukan atau format teksnya rusak.

2.14. Kelas CommandHandler

Kelas ini berada pada User Interaction Layer yang bertindak sebagai jembatan (atau Controller) antara masukan teks yang diketik oleh pengguna di terminal dengan state permainan di kelas Game. Tanggung jawab utamanya adalah mendengarkan input, memotong (parsing) string perintah beserta argumennya (misal ATUR_DADU 2 5), memvalidasi apakah perintah tersebut bisa dipanggil saat itu, dan mengeksekusi method logika yang sesuai di kelas Game.

Atribut:

- game : IGameAction& → referensi ke abstract base class permainan sebagai pintu masuk akses ke Game.
- renderer : CLIRenderer& → referensi ke kelas perender untuk langsung menampilkan balasan/hasil operasi.

Method:

- listen() : void → loop utama untuk meminta input teks (>) dari pengguna saat giliran berlangsung.
- handleInput(rawInput : string) → handle setiap satu baris input
- promptInput(prompt: string) → minta input konfirmasi

Relasi:

- Berasosiasi kuat dengan kelas Game. CommandHandler hanya memanggil fungsi, dan tidak boleh melakukan kalkulasi logika permainan (seperti menghitung uang) di dalam dirinya sendiri.
- Berasosiasi dengan CLIRenderer untuk mencetak output ke layar terminal.

2.15. Kelas CommandParser

Kelas ini berada pada User Interaction Layer yang mengubah input teks mentah dari pengguna menjadi objek perintah yang terstruktur. Tanggung jawab utamanya adalah melakukan tokenisasi string, memvalidasi jumlah serta tipe data argumen (seperti memastikan koordinat dadu adalah angka), dan menginstansiasi kelas *Command* yang spesifik (seperti LemparDaduCommand atau BangunCommand) untuk kemudian dieksekusi oleh sistem.

Method:

- tokenize(const std::string& input) : static std::vector<std::string> → memecah string berdasarkan spasi dan mengubah setiap kata menjadi huruf kapital untuk memudahkan pencocokan perintah.
- requireArgCount(const std::vector<std::string>& tokens, int expected, const std::string& cmd) : static void → memastikan jumlah argumen yang dimasukkan pengguna sesuai dengan kebutuhan perintah.
- parseIntArg(const std::string& token, const std::string& cmd) : static int → mengonversi string argumen menjadi integer.
- parse(const std::string& rawInput) : static Command* → method utama yang memetakan kata pertama (kata kunci perintah) ke pembuatan objek Command yang relevan.

Relasi:

- Digunakan oleh child class Command dan CommandHandler

2.16. Kelas CLIRenderer

Kelas ini juga berada di dalam User Interaction Layer dan didedikasikan murni untuk urusan visualisasi atau pencetakan teks (View). Tanggung jawab utamanya adalah mengambil data status (state) dari kelas-kelas model (seperti Board, Tile, Player) dan memformatnya menjadi tampilan visual yang rapi di terminal, termasuk pewarnaan blok lahan, tabel legenda, dan panel khusus.

Atribut:

- Tidak memiliki atribut state yang persisten, namun bisa menyimpan konstanta warna (ANSI Escape Codes).

Method:

- renderBoard(board : Board&, players : vector<Player*>, currentTurn, maxTurn, currentPlayerIndex) : void → memproses indeks setiap petak di papan dan posisi bidak semua pemain untuk menggambar kerangka persegi papan permainan Nimonspoli secara terstruktur, lengkap dengan colored print sesuai color group, indikator kepemilikan dan bangunan, posisi bidak, serta panel legenda dan informasi turn di tengah papan. Dipanggil saat perintah CETAK_PAPAN.
- printDeed(tile : PropertyTile&) : void → menampilkan format tabel Akta Kepemilikan yang mendetail (harga beli, sewa berbagai level, harga rumah/hotel, dan status) untuk perintah CETAK_AKTA.
- printPropertyInventory(player : Player&) : void → menampilkan aset pemain dikelompokkan berdasarkan warna (color group) untuk perintah CETAK_PROPERTI.
- printPlayerStatus(player: Player&, board : Board&) : void → menampilkan ringkasan status pemain aktif mencakup saldo, posisi petak saat ini, status (ACTIVE/JAILED), total kekayaan, status shield dan diskon aktif, serta daftar kartu kemampuan di tangan. Dipanggil saat perintah CETAK_STATUS.
- printMainMenu() : void → menampilkan header dan menu awal
- printTurnheader(player: Player&, turn : int, maxTurn : int) : void → menampilkan header giliran
- printPrompt() : void → nampilin kalimat-kalimat prompt
- printHelp() : void → nampilin daftar perintah
- printError(message : string) : void → mencetak pesan error ke stderr dengan warna merah menggunakan ANSI color code.
- printInfo(message : string) : void → mencetak narasi kejadian standar ke stdout
- printSuccess(message : string) : void → mencetak pesan sukses ke stdout dengan warna hijau.
- printWinner(winner : Player&, turn : int) : void → menampilkan layar akhir permainan dengan informasi pemenang, total kekayaan, dan turn penyelesaian.
- getColorCode(colorGroup : string) : string → helper buat dapetin colorgroup
- getColorTag(colorGroup : string) : string → helper buat dapetin tag colorgroup
- promptDropCard(cardNames : vector<string>) : int → menampilkan daftar kartu di tangan pemain dan meminta pemain memilih kartu yang akan dibuang saat slot penuh.
- promptChoice() : int → membaca pilihan numerik dari pengguna untuk digunakan pada panel interaktif seperti likuidasi.

- promptTileCode() : string → meminta pengguna memasukkan kode petak, digunakan untuk fitur TeleportCard dan CETAK_AKTA.
- promptPropertyChoice(props : vector<PropertyTile>) → menampilkan daftar properti beserta nilai gadainya dan meminta pemain memilih nomor properti.
- promptYesNo(question: string) : bool → menampilkan pertanyaan konfirmasi dan membaca input y/n dari pengguna, mengembalikan true/false.

Relasi:

- Menerima referensi data secara read-only (seringkali dilewatkan menggunakan const reference) dari Board, Player, dan Tile murni hanya untuk keperluan pembacaan data.
- Dipanggil oleh CommandHandler maupun event dari Game ketika ada antarmuka spesifik yang perlu ditampilkan ke layar.

2.17. Kelas IGameAction

Kelas ini menjadi interface yang menghubungkan Command dengan logika di Game. Supaya tidak terjadi circular dependency dan menjaga enkapsulasi, digunakan kelas IGameAction ini yang berinteraksi dengan Command, dan diimplementasikan di kelas Game (.hpp dan .cpp).

Atribut:

- Tidak ada atribut khusus

Method:

- getActivePlayers() : void (pure virtual) → mendapatkan informasi pemain yang memiliki status aktik (tidak dipenjara)
- processTileLandingPublic(player : Player&, tileIndex : int) : void (pure virtual) → menangani logic ketika player mendarat di suatu petak
- setGameActive(active : bool) : void (pure virtual) → merubah state permainan menjadi aktif atau tidak
- isGameActive() : void (pure virtual) → status permainan sedang berjalan atau tidak
- getBoard() : Board& (pure virtual) → mendapatkan informasi terkait papan permainan
- getCurrentPlayer() : Player* (pure virtual) → mendapatkan informasi pemain yang sedang giliran saat ini
- rolleDice() : void (pure virtual) → melempar dadu secara acak

- setDice(x : int, y : int) : void (pure virtual) → melempar dadu dengan nilai yang diatur secara manual
- mortgageProperty(tileCode : string) : void (pure virtual) → menggadaikan properti
- redeemProperty(tileCode : string) : void (pure virtual) → menebus properti
- buildOnProperty(tileCode : string) : void (pure virtual) → membangun rumah atau hotel di properti
- sellAllBuildingsInGroup(colorGroup : string) : void (pure virtual) → menjula seluruh bangunan di properti sebagai syarat gadai
- placeBid(amount : int) : void (pure virtual) → memberikan tawaran dalam lelang
- passAuction() : void (pure virtual) → lewati lelang
- finalizeAuction(winner : Player*, property : PropertyTile*, winningBid : int) : void (pure virtual) → kondisi akhir lelang
- logAuctionEvent(action : string, detail : string) : void (pure virtual) → log aktivitas lelang
- triggerAuction(property : PropertyTile&) : void (pure virtual) → memulai lelang
- useSkillCard(cardIndex : int) : void (pure virtual) → menggunakan kartu kemampuan
- applyMoveCard(steps : int) : void (pure virtual) → menggunakan Move Card
- applyTeleportCard() : void (pure virtual) → menggunakan Teleport Card
- applyDiscountCard(percent : int) : void (pure virtual) → menggunakan Discount Card
- applyShieldCard() : void (pure virtual) → menggunakan Shield Card
- applyLassoCard() : void (pure virtual) → menggunakan Lasso Card
- applyDemolitionCard() : void (pure virtual) → menggunakan Demolition Card
- payJailFine() : void (pure virtual) → membayar denda penjara
- useJailFreeCard() : void (pure virtual) → menggunakan kartu bebas penjara
- saveGame(filename : string) : void (pure virtual) → menyimpan progres game dalam sebuah file.txt
- loadGame(filename : string) : void (pure virtual) → memuat progres game dari sebuah file.txt
- printBoard() : void (pure virtual) → menampilkan papan permainan ke terminal
- printDeed(tileCode: string) : void (pure virtual) → menampilkan akta properti ke terminal
- printPropertyInventory() : void (pure virtual) → menampilkan properti yang dimiliki pemain ke terminal
- printLog(limit : int) : void (pure virtual) → menampilkan log yang sudah tercatat
- printHelp() : void (pure virtual) → menampilkan daftar perintah
- printPlayerStatus() : void (pure virtual) → menampilkan status pemain saat ini
- getLogger() : void (pure virtual) → mendapatkan log aktivitas yang sudah tercatat

- countActivePlayers() : void (pure virtual) → menghitung banyak pemain aktif
- getCurrentPlayerName() : void (pure virtual) → mendapatkan username pemain yang sedang bermain saat ini
- endTurn() : void (pure virtual) → mengakhiri giliran pemain saat ini
- declareBankruptcy() : void (pure virtual) → pemain dinyatakan kalah dan tidak aktif lagi

Relasi:

- Digunakan oleh Command untuk memanggil fungsi-fungsi yang ada di Game

3. Justifikasi Desain

Desain kelas pada sistem Nimonspoli menggunakan pendekatan Layered Architecture yang memisahkan tanggung jawab ke dalam tiga lapisan. Data Access Layer mencakup ConfigLoader, SaveLoadManager, dan TransactionLogger yang bertugas murni pada operasi baca-tulis file. Game Logic Layer mencakup hierarki Tile, Player, Card, Dice, AuctionManager, BankruptcyManager, dan Game sebagai inti logika permainan. User Interaction Layer mencakup CommandHandler dan CLIRenderer yang menangani input dari pengguna dan output ke terminal. Pemisahan ini memastikan perubahan pada satu lapisan tidak berdampak ke lapisan lain. Misal contoh terdapat perubahan format output, maka Game Logic tidak perlu disentuh sama sekali.

Hierarki Tile dirancang menggunakan abstract class dengan pure virtual function onLanded() supaya Board cukup menyimpan vector<Tile*> dan memanggil aksi tiap petak secara polimorfik tanpa perlu tahu tipe konkretnya. Pendekatan ini juga memudahkan penambahan jenis petak baru di masa depan tanpa perlu mengubah Board maupun Game. Hal yang sama diterapkan pada hierarki ActionCard dan SkillCard dengan pure virtual function execute() dan use().

CardDeck dibuat sebagai generic class CardDeck<T> karena ketiga jenis tumpukan kartu dalam permainan punya perilaku yang identik secara struktural. Tanpa generic class, ketiganya harus diimplementasikan secara terpisah dengan kode yang hampir sama persis, yang melanggar prinsip DRY.

Tanggung jawab Game dipecah dengan mengekstrak AuctionManager dan BankruptcyManager sebagai kelas tersendiri. Mekanisme lelang dan kebangkrutan di spesifikasi cukup kompleks sehingga kalau keduanya ditangani langsung di Game, kelas tersebut akan terlalu besar dan sulit dipelihara. Dengan pemisahan ini, Game tetap berperan sebagai orkestrator tanpa menanggung logika yang terlalu spesifik.

Operator overloading diterapkan di kelas Player untuk dua keperluan: operator+= dan operator-= untuk transaksi uang, serta operator< dan operator> untuk perbandingan total kekayaan antar pemain saat menentukan pemenang di kondisi MAX TURN.

Terkait kelebihan dan kekurangan desain kelas yang kami implementasikan, berikut penjabarannya.
Kelebihan:

- Layered Architecture membuat setiap lapisan bisa dikerjakan dan diuji secara independen, yang sangat membantu dalam pengerjaan kelompok karena pembagian tugas bisa dilakukan per layer.
- Penggunaan abstract class dan polymorphism pada hierarki Tile membuat sistem mudah dikembangkan.
- CardDeck<T> sebagai generic class menghindari duplikasi kode untuk tiga jenis deck yang berbeda.
- Pemisahan AuctionManager dan BankruptcyManager dari Game membuat masing-masing mekanisme lebih mudah di-debug dan diuji secara terpisah.

Kekurangan:

- Beberapa method di Game masih berpotensi cukup panjang karena tetap harus mengorkestrasi banyak komponen sekaligus dalam satu giliran pemain.
- Penggunaan raw pointer Tile* di Board memerlukan manajemen memori manual yang rawan bug jika tidak dikelola dengan hati-hati saat implementasi.
- Hierarki kelas yang cukup dalam, terutama di sisi Tile, bisa membuat alur kode lebih susah ditelusuri saat debugging karena harus melacak pemanggilan virtual function ke subclass yang tepat.

Kendala utama yang dialami dalam pemilihan desain adalah proses abstraksi dari spesifikasi ke kelas-kelas yang dibutuhkan. Spesifikasi Nimonspoli ditulis dari sudut pandang mekanisme permainan, bukan dari sudut pandang teknis, sehingga menentukan kelas mana yang perlu ada, apa tanggung jawabnya, dan di mana batas antar kelas cukup membutuhkan banyak pertimbangan. Misalnya, mekanisme bangkrut di spesifikasi mencakup likuidasi aset, transfer ke kreditor, dan lelang ulang ke Bank dalam satu alur yang panjang. Pada kasus ini, menentukan apakah ini tanggung jawab Game, BankruptcyManager, atau keduanya bukan keputusan yang langsung jelas.

Kendala lain muncul pada penentuan batas antara Game Logic Layer dan User Interaction Layer. Beberapa aksi seperti proses lelang dan likuidasi melibatkan interaksi bolak-balik dengan pemain (input bid, pilih properti untuk dijual) sehingga cukup sulit menentukan bagian mana yang murni logika dan bagian mana yang termasuk interaksi dengan pengguna.

4. Penerapan Konsep OOP

4.1. Inheritance & Polymorphism

a. Kelas Tile dan Kelas Turunannya

Tile sebagai abstract base class. PropertyTile, ActionTile, dan SpecialTile mewarisi Tile.StreetTile, RailroadTile, UtilityTile mewarisi PropertyTile.CardTile, FestivalTile, TaxTile mewarisi ActionTile. GoTile, JailTile, FreeParkingTile, dan GoToJailTile mewarisi SpecialTile. Hierarki Tile menerapkan konsep inheritance karena petak-petak pada papan permainan Nimonspoli memiliki atribut dasar yang sama, yaitu indeks, kode, dan nama, serta perilaku umum berupa onLanded() yang dipanggil saat pemain mendarat. Kelas Tile dijadikan base class agar seluruh jenis petak dapat diperlakukan secara seragam sebagai Tile*, sementara detail perilaku petak ditangani oleh masing-masing dalam subkelasnya. Polymorphism diterapkan melalui fungsi virtual calculateRent() pada kelas PropertyTile. Kelas Board atau Game cukup menyimpan petak sebagai Tile*. Lalu saat membayar sewa, kelas tersebut hanya perlu mengubah Tile menjadi PropertyTile untuk memanggil calculateRent() yang implementasinya berbeda untuk tiap properti.

Cuplikan Kode Kelas Tile Abstrak:

```
class Tile {
    protected:
        int index; // posisi tile dalam board
        std::string code; // kode unik
        std::string name; // nama
    public:
        // constructor
        Tile(int idx, std::string cd, std::string nm);
        // destructor
        virtual ~Tile() = default;
        // dipanggil saat player mendarat
        virtual void onLanded(Player& player, Game& game) = 0;
```

Cuplikan Kode Kelas ActionTile Mewarisi Tile:

```
// tetapi memiliki efek tertentu saat dilinjak
class ActionTile: public Tile {
    public:
        // ctor
        ActionTile(int idx, std::string cd, std::string nm);
        // getter tile type (return ACTION)
        TileType getTileType() const override;
```

Cuplikan Kode Kelas SpecialTile Mewarisi Tile:

```
// merepresentasikan petak spesial yang terletak di 4 sudut pap
class SpecialTile : public Tile {
    public:
        // ctor
        SpecialTile(int idx, std::string cd, std::string nm);
        // getter tile type (return SEPCIAL)
        TileType getTileType() const override;
```

Cuplikan Kode Kelas PropertyTile Mewarisi Tile:

```
class PropertyTile : public Tile {
    protected:
        int buyPrice; // harga beli properti
        int mortgageValue; // nilai uang yang diterima saat properti digadai
        Player* owner; // pointer ke player pemilik properti
        PropertyStatus status; // status properti (bank, owned, mortgaged)
```

Cuplikan Kode Sebuah Fungsi pada Game Memanggil calculateRent() PropertyTile*:

```
// Helper buat trigger sewa
void Game::applyRent(Player& player, PropertyTile& tile) {
    if (player.isShieldActive()) {
        renderer->printInfo("[SHIELD ACTIVE] Kamu terlindungi dari sewa!");
        return;
    }

    int rent = tile.calculateRent(dice->getTotal());
    Player* owner = tile.getOwner();
```

b. Kelas Card dan Kelas Turunannya

Kelas Card dan turunannya (ActionCard dan SkillCard) menggunakan konsep inheritance karena semua objek tersebut adalah kartu, namun dengan perilaku yang berbeda-beda. Setiap kartu bisa punya perilaku berbeda, tetapi tetap dipanggil lewat fungsi yang sama, misalnya execute(). Dari sisi reusability, atribut umum seperti name dan description cukup didefinisikan sekali di kelas Card. Semua turunan langsung mewarisinya, sehingga tidak perlu menulis ulang kode yang sama di banyak tempat. Selain itu, desain ini mendukung pengembangan ke depan. Jika ingin menambah jenis kartu baru, cukup buat subclass baru tanpa harus mengubah struktur utama yang sudah ada.

Cuplikan Kode Kelas Card:

```
class Card {
protected:
    std::string name;        // Nama dari kartu
    std::string description; // Deskripsi efek kartu

public:
    Card(std::string n, std::string d) : name(n), description(d) {}
    virtual ~Card() = default;

    std::string getName() const { return name; }               // Getter nama kartu
    std::string getDescription() const { return description; } // Getter deskripsi kartu
};
```

Cuplikan Kode Kelas ActionCard:

```
class ActionCard : public Card {
protected:
    CardType cardType;
public:
    ActionCard(std::string n, std::string d, CardType t) : Card(n, d), cardType(t) {}

    // Mengeksekusi instruksi instan kepada pemain
    virtual void execute(Player& player, Game& game) = 0;

    CardType getCardType() const { return cardType; }
};

// --- Subkelas Kesempatan ---

class GoToNearestStationCard : public ActionCard {
public:
    GoToNearestStationCard();
    void execute(Player& player, Game& game) override;
};

class MoveBackCard : public ActionCard {
public:
    MoveBackCard();
    void execute(Player& player, Game& game) override;
};
```

Cuplikan Kode Kelas SkillCard:

```
class SkillCard : public Card {
public:
    SkillCard(std::string n, std::string d) : Card(n, d) {}
};

class MoveCard : public SkillCard {
private:
    int steps; // Jumlah langkah maju
public:
    MoveCard(int s);
    int getValue() const { return steps; }
};

class DiscountCard : public SkillCard {
private:
    int percent;  // Persentase diskon
    int duration; // Sisa durasi (1 giliran)
public:
    DiscountCard(int p, int dur);
    int getValue() const { return percent; }
    int getDuration() const { return duration; }
};
```

c. Kelas IGameAction dan Kelas Game

IGameAction dibuat sebagai abstract interface karena ada dua kebutuhan yang berbeda. Dari sisi Command, setiap command seperti LemparDaduCommand, GadaiCommand, dll butuh memanggil aksi game. Tapi command tidak boleh bergantung langsung ke Game yang konkret, karena itu menciptakan coupling ketat antara UI layer dan game logic layer. Dengan IGameAction, command cukup tau "game bisa melakukan ini" tanpa peduli implementasi detailnya. Sementara itu dari sisi Game, Game mengimplementasikan IGameAction karena dia memang satu-satunya implementasi konkret yang menjalankan semua aturan permainan. Dengan mewarisi IGameAction, Game dipaksa mengimplementasikan semua kontrak yang dibutuhkan command sehingga kalau ada method yang belum diimplementasi, compiler langsung error.

Keuntungan yang didapat dari implementasi seperti ini antara lain modularitas dalam pengembangkan. Misalkan suatu saat di masa depan ingin membuat suatu class baru, maka tinggal mengimplementasikan IGameAction tanpa harus merubah satu pun command. Keuntungan lainnya yaitu command layer dan game layer bisa dikembangkan secara independen selama kontrak IGameAction terpenuhi.

Cuplikan Kode Kelas IGameAction.

```
class IGameAction
{
public:
    virtual ~IGameAction() = default;

    virtual std::vector<Player*> getActivePlayers() const = 0;
    virtual void processTileLandingPublic(Player& player, int tileIndex) = 0;

    virtual void setGameActive(bool active) = 0;
    virtual bool isGameActive() const = 0;
    virtual Board &getBoard() const = 0;
    virtual Player *getCurrentPlayer() const = 0;

    // Dadu
    virtual void rollDice() = 0;
    virtual void setDice(int x, int y) = 0;

    // Property
    virtual void mortgageProperty(const std::string& tileCode) = 0;
    virtual void redeemProperty(const std::string &tileCode) = 0;
    virtual void buildOnProperty(const std::string &tileCode) = 0;
    virtual void sellAllBuildingsInGroup(const std::string& colorGroup) = 0;
```

Cuplikan Kode Kelas Game

```
public:
    // Konstruktor dan destruktor
    Game();
    ~Game() override;

    // Inisialisasi awal
    void initialize();
    void startNewGame(const std::vector<std::string>& playerNames);
    bool isGameActive() const override;
    void setGameActive(bool active) override;

    // Dadu
    void rollDice() override;
    void setDice(int x, int y) override;

    // Property
    void mortgageProperty(const std::string& tileCode) override;
    void redeemProperty(const std::string &tileCode) override;
    void buildOnProperty(const std::string &tileCode) override;
    void sellAllBuildingsInGroup(const std::string& colorGroup) override;

    // Auction
    void placeBid(int amount) override;
    void passAuction() override;
    void finalizeAuction(Player* winner, PropertyTile* property, int winningBid) override;
    void logAuctionEvent(const std::string& action, const std::string& detail) override;
    void triggerAuction(PropertyTile& property) override;
```

Cuplikan Kode Kelas Command: AkhiriGiliranCommand

```cpp
#pragma once
#include "Command.hpp"
#include "core/IGameAction.hpp"
#include <string>

// AKHIRI_GILIRAN
class AkhiriGiliranCommand : public Command {
public:
    bool execute(IGameAction& game) override {
        // Panggil fungsi akhiri giliran yang ada di kelas Game
        game.endTurn();
        return false;
    }

    std::string getName() const override {
        return "AKHIRI_GILIRAN";
    }
};
```

d. Kelas NimonspoliException

Kelas Exception Handling khusus yang berfungsi menangani berbagai galat dan kondisi tak valid agar program tidak berhenti tiba-tiba. Kelas NimonspoliException digunakan untuk mengembalikan sebuah error apabila suatu syarat tercapai, yakni ketika suatu nilai tidak temasuk ke dalam parameter ideal fungsinya. Kelas ini merupakan base class yang mewarisi std::exception.

```cpp
#include "../../include/utils/NimonspoliException.hpp"

NimonspoliException::NimonspoliException(const std::string& message)
    : message(message) {}

const char* NimonspoliException::what() const noexcept {
    return message.c_str();
}
```

a. Kelas InsufficientFundsException

Exception ini dilempar ketika seorang pemain mencoba melakukan transaksi (membeli properti, membangun rumah, membayar denda, atau menawar lelang) namun saldo cash yang dimilikinya tidak mencukupi. Exception ini digunakan untuk menangani kasus dimana cash sebuah player tidak cukup untuk melakukan suatu aksi, seperti membeli properti, mengurangi saldo, dan membayar sewa.

```
InsufficientFundsException::InsufficientFundsException(const std::string& message)
    : NimonspoliException(message) {}
```

b. Kelas InvalidActionException

Exception ini dilempar ketika pemain mencoba melakukan aksi yang melanggar logika atau aturan permainan. Exception ini digunakan untuk menangani kasus dimana sebuah metode Action yang dilakukan pemain melanggar suatu kondisi atau ketentuan permainan.

```
InvalidActionException::InvalidActionException(const std::string& message)
    : NimonspoliException(message) {}
```

c. Kelas CardSlotFullException

Exception ini dilempar ketika pemain mendapatkan kartu baru namun jumlah kartu yang dimilikinya sudah mencapai batas maksimal, yaitu 3 kartu. Exception ini digunakan untuk menangani kasus dimana ada metode yang memasukkan actionCard atau skillCard ke dalam Card Slot sedangkan Card Slot sudah penuh.

```
CardSlotFullException::CardSlotFullException(const std::string& message)
    : NimonspoliException(message) {}
```

d. Kelas PropertyNotOwnedException

Exception ini dilempar ketika ada operasi yang dilakukan pada properti yang statusnya bukan milik pemain yang bersangkutan. Exception ini digunakan untuk menangani kasus dimana ada metode yang memerlukan properti namun pemain yang memanggil metode ini bukan pemilik properti saat itu.

```
PropertyNotOwnedException::PropertyNotOwnedException(const std::string& message)
    : NimonspoliException(message) {}
```

e. Kelas FileNotFoundException

Exception ini dilempar ketika sistem gagal menemukan file .txt yang diperlukan. Exception ini digunakan untuk menangani kasus dimana ketika memulai game dari Save File, File input tidak ditemukan atau tidak dapat dibaca.

```
FileNotFoundException::FileNotFoundException(const std::string& message)
    : NimonspoliException(message) {}
```

f. Kelas InvalidInputException

Exception ini dilempar ketika input teks dari pengguna tidak sesuai dengan format yang dikenali oleh sistem. Exception ini digunakan untuk menangani kasus dimana input yang dimasukkan tidak sesuai dengan parameter metode yang diharapkan.

```
InvalidInputException::InvalidInputException(const std::string& message)
    : NimonspoliException(message) {}
```

4.2. Method/Operator Overloading

a. Kelas Player

Kelas Player memanfaatkan operator overloading pada beberapa metode, yakni

1. Operator+=

   Operator ini digunakan untuk menambahkan jumlah cash yang dimiliki suatu player dengan sebuah besaran integer. Apabila besaran integer berjumlah negatif, metode akan mengeluarkan error. Berikut cuplikan kode pada metode operator+= kelas player.

```
Player &Player::operator+=(int amount)
{
    if (amount < 0)
    {
        throw NimonspoliException("[Operator+=] Tidak boleh menggunakan jumlah negatif");
    }
    cash += amount;
    return *this;
}
```

2. Operator-=

   Operator ini digunakan untuk mengurangi jumlah cash yang dimiliki suatu player dengan sebuah besaran integer. Apabila besaran integer melebihi jumlah cash pada player atau besaran integer berjumlah negatif, metode akan mengeluarkan error. Berikut cuplikan kode pada metode operator-= kelas player.

```
Player &Player::operator-=(int amount)
{
    if (amount < 0)
    {
        throw NimonspoliException("[Operator-=] Tidak boleh menggunakan jumlah negatif");
    }
    if (cash - amount < 0)
    {
        throw NimonspoliException("[Operator-=] Uang tidak cukup untuk melakukan transaksi");
    }
    cash -= amount;
    return *this;
}
```

4.3. Template & Generic Classes

a. Generic Class CardDeck<T>

Penggunaan generic class pada CardDeck<T> dalam tugas besar ini merupakan penerapan prinsip DRY (Don't Repeat Yourself). Pada sistem yang kami rancang terdapat beberapa jenis (kelas) kartu yang berbeda. Meskipun jenis kartunya berbeda, mekanisme dasar card deck-nya sama. Tanpa generic class, akan diperlukan berbagai kelas sesuai jenis kartu secara terpisah (misal SkillDeck, ChanceDeck, dll). Dengan adanya generic class mekanisme card deck dapat ditulis dalam satu kelas, yaitu CardDeck<T>.

```cpp
#ifndef CARDDECK_HPP
template <typename T>
class CardDeck {
private:
    std::queue<T*> cards;             // Antrean kartu yang siap ditarik
    std::vector<T*> discardPile;      // Tumpukan kartu yang sudah terpakai (dijadikan vector untuk mempermudah shuffle ulang)

public:
    CardDeck() = default;

    // Destructor untuk memastikan manajemen memori yang aman
    ~CardDeck() {
        while (!cards.empty()) {
            delete cards.front();
            cards.pop();
        }
        for (T* card : discardPile) {
            delete card;
        }
    }

    // Mengambil kartu teratas. Jika habis, kocok ulang dari discardPile
    T* draw() {
        if (cards.empty()) {
            if (discardPile.empty()) {
                return nullptr; // Kondisi deck sepenuhnya kosong
            }

            // Reshuffle dari discardPile menjadi cards baru
            std::random_device rd;
            std::mt19937 g(rd());
            std::shuffle(discardPile.begin(), discardPile.end(), g);
```

4.4. Exception

a. Kelas NimonspoliException

Kelas NimonspoliExceptiom digunakan untuk mengembalikan sebuah error apabila suatu syarat tercapai, yakni ketika suatu nilai tidak temasuk ke dalam parameter ideal fungsinya. Berikut cuplikan kelas NimonspoliException:

```
NimonspoliException::NimonspoliException(const std::string &message)
    : message(message) {}

const char *NimonspoliException::what() const noexcept
{
    return message.c_str();
}
```

Beberapa Kelas NimonspoliException menangani beberapa penanganan error berdasarkan syarat penangkapannya, yakni:

1. InsufficientFundsException
   Exception ini digunakan untuk menangani kasus dimana cash sebuah player tidak cukup untuk melakukan suatu aksi, seperti membeli properti, mengurangi saldo, dan membayar sewa. Berikut cuplikan penggunaannya:

```
Player &Player::operator-=(int amount)
{
    if (amount < 0)
    {
        throw InvalidInputException("[Operator-=] Tidak boleh menggunakan jumlah negatif");
    }
    if (cash - amount < 0)
    {
        throw InsufficientFundsException("[Operator-=] Uang tidak cukup untuk melakukan transaksi");
    }
    cash -= amount;
    return *this;
}
```

2. InvalidActionException
   Exception ini digunakan untuk menangani kasus dimana sebuah metode Action yang dilakukan pemain melanggar suatu kondisi atau ketentuan permainan. Berikut cuplikan penggunaannya:

```
void Player::addCard(SkillCard *card)
{
    if (card == nullptr)
    {
        throw InvalidActionException("[addCard] Kartu tidak boleh null");
    }
    ownedSkillCards.push_back(card);
}
```

3. CardSlotFullException
   Exception ini digunakan untuk menangani kasus dimana ada metode yang memasukkan actionCard/skillCard ke dalam Card Slot sedangkan Card Slot sudah penuh.

4. PropertyNotOwnedException
   Exception ini digunakan untuk menangani kasus dimana ada metode yang memerlukan properti namun pemain yang memanggil metode ini bukan pemilik properti saat itu. Berikut cuplikan penggunaannya:

5. FileNotFoundException
   Exception ini digunakan untuk menangani kasus dimana ketika memulai game dari Save File, File input tidak ditemukan atau tidak dapat dibaca.

6. InvalidInputException
   Exception ini digunakan untuk menangani kasus dimana Input yang dimasukkan tidak sesuai dengan parameter metode yang diharapkan. Berikut cuplikan penggunaannya:

```
// operator overloads
Player &Player::operator+=(int amount)
{
    if (amount < 0)
    {
        throw InvalidInputException("[Operator+=] Tidak boleh menggunakan jumlah negatif");
    }
    cash += amount;
    return *this;
}
```

,

4.5. C++ Standard Template Library

a. std::vector dan std::map di Kelas Tile

Cuplikan Kode UtilityTile dengan Atribut multiplierTable Bertipe std::map

```
class UtilityTile : public PropertyTile {
    private:
        std::map<int, int> multiplierTable; // pemetaan jumla
    public:
```

Cuplikan Kode StreetTile dengan Atribut rents Bertipe std::vector

```
class StreetTile : p
    private:
        std::string colorGroup; // kelompok warna
        std::vector<int> rents; // tabel sewa level
```

Cuplikan Kode Railroad Tile dengan Atribut rentTable Bertipe std::vector

```
class RailroadTile : public PropertyTile {
    private:
        std::map<int, int> rentTable; // pemetaan
```

Kelas StreetTile, RailroadTile, dan Utility tile menggunakan STL containers pada atributnya. STL containers ini berupa map dan vector. Penggunaan STL ini memungkinkan penggunaan fungsi manipulasi elemen bawaan tanpa perlu mendefinisikan manual. Proses inisialisasi dan penghancuran container juga telah ditangani oleh STL tersebut sehingga penggunaanya praktis.

b. std::queue di Kelas Generik CardDeck

Alasan pemilihan penggunaan std::queue adalah karena operasi utama deck kartu hanya dua, yaitu draw() dari depan dan discard() ke tumpukan buangan. Queue secara natural mendukung operasi ini dengan kompleksitas O(1) untuk keduanya, tanpa perlu menggeser elemen seperti yang terjadi di vector. Kalau pakai vector, setiap draw() harus hapus elemen pertama dengan erase(begin()) yang O(n) karena semua elemen bergeser. Queue menghindari overhead ini. Hal tersebut juga merupakan kelebihan dari queue yang dijadikan pertimbangan dalam penentuan design penggunaan.

Cuplikan Kode Kelas Generik CardDeck

```
template <typename T>
class CardDeck {
private:
    std::queue<T*> cards;            // Antrean kartu yang siap ditarik
    std::vector<T*> discardPile;     // Tumpukan kartu yang sudah terpakai (dijadikan vector untuk mempermudah shuffle ulang)
```

c. std::unique_ptr di Game

Alasan pemilihan unique_ptr selain karena direkomendasikan oleh Kak El ketika asistensi, komponen-komponen utama seperti board, dice, logger, bankruptcymanager, auctionamanger, dan semua deck kartu memiliki lifetime yang terikat langsung ke Game sehingga saat Game dihancurkan, semua komponen ini juga harus ikut dihancurkan. unique_ptr menjamin hal ini secara otomatis tanpa perlu memanggil delete secara manual di destruktor, menghindari potensi memory leak. Selain itu, unique_ptr memastikan hanya satu owner yang boleh memegang pointer ini pada satu waktu. Ini sesuai karena memang tidak ada komponen lain yang boleh ikut memiliki Board atau Dice, di mana hanya Game yang bertanggung jawab atas keduanya.

Kelebihan dari penggunaan unique_ptr antara lain compiler secara otomatis men-generate destruktor yang membersihkan semua unique_ptr. Dengan begitu, tidak diperlukan delete secara manual yang rawan lupa dan menyebabkan memory leak.

```
// ===== LOAD =====
void Game::loadGame(const std::string& filename) {
    std::string fullPath = "saves/" + filename;
    SaveLoadManager slm;
    std::vector<Player*> rawPlayers;
    slm.loadGame(fullPath, turnsPlayed, maxTurn, rawPlayers, turnOrder, currentPlayerIndex, *board, *skillCardDeck, *logger);

    // Pindahkan raw pointer ke unique_ptr
    players.clear();
    for (Player* p : rawPlayers) {
        players.push_back(std::unique_ptr<Player>(p));
    }

    logger->addLog("Permainan dimuat dari: " + filename);
}
```

d. std::sort di Game

Alasan pemilihan std::sort pada method handleGameEnd() di Game adalah karena penentuan pemenang membutuhkan pengurutan multi-kriteria, yaitu pertama harus berdasarkan uang terbanyak, kalau seri berdasarkan jumlah properti terbanyak, kalau masih seri berdasarkan jumlah kartu terbanyak. std::sort dengan custom comparator lambda memungkinkan semua kriteria ini diekspresikan dalam satu blok kode yang ringkas:

```
// Urutan menang: uang terbanyak, properti terbanyak, kartu terbanyak
std::sort(active.begin(), active.end(), [](Player* a, Player* b) {
    if (a->getCash() != b->getCash()) return a->getCash() > b->getCash();
    if (a->getOwnedProperties().size() != b->getOwnedProperties().size())
        return a->getOwnedProperties().size() > b->getOwnedProperties().size();
    return a->getOwnedSkillCards().size() > b->getOwnedSkillCards().size();
});
```

Kelebihan dari penggunaan std::sort pada kasus ini yaitu std::sort menggunakan algoritma introsort yang kompleksitasnya O(n log n). Algoritma tersebut optimal untuk kasus ini karena jumlah pemain maksimal 4, tapi tetap efisien kalau suatu saat diperluas. Lambda comparator juga membuat logika pengurutan multi-kriteria terbaca jelas dan terlokalisasi di satu tempat, dibanding harus menulis fungsi comparator terpisah.

4.6. Konsep OOP lain

a. Abstract Base Class

i. Abstract Base Class Tile

Cuplikan Kode Kelas Tile Abstrak

```
class Tile {
    protected:
        int index; // posisi tile dalam board
        std::string code; // kode unik
        std::string name; // nama
    public:
        // constructor
        Tile(int idx, std::string cd, std::string nm);
        // destructor
        virtual ~Tile() = default;
        // dipanggil saat player mendarat
        virtual void onLanded(Player& player, Game& game) = 0;
```

Tile berperan sebagai abstract base class karena memiliki virtual function onLanded() yang menjadi kontrak bagi seluruh subclass. Dengan desain ini, setiap jenis petak wajib menyediakan implementasi perilaku/efek ketika pemain mendarat.

ii. Abstract Base Class IGameAction

```
class IGameAction
{
public:
    virtual ~IGameAction() = default;

    virtual std::vector<Player*> getActivePlayers() const = 0;
    virtual void processTileLandingPublic(Player& player, int tileIndex) = 0;

    virtual void setGameActive(bool active) = 0;
    virtual bool isGameActive() const = 0;
    virtual Board &getBoard() const = 0;
    virtual Player *getCurrentPlayer() const = 0;

    // Dadu
    virtual void rollDice() = 0;
    virtual void setDice(int x, int y) = 0;

    // Property
    virtual void mortgageProperty(const std::string& tileCode) = 0;
    virtual void redeemProperty(const std::string &tileCode) = 0;
    virtual void buildOnProperty(const std::string &tileCode) = 0;
    virtual void sellAllBuildingsInGroup(const std::string& colorGroup) = 0;

    // Auction
    virtual void placeBid(int amount) = 0;
    virtual void passAuction() = 0;
    virtual void finalizeAuction(Player* winner, PropertyTile* property, int winningBid) = 0;
    virtual void logAuctionEvent(const std::string& action, const std::string& detail) = 0;
    virtual void triggerAuction(PropertyTile& property) = 0;

    // SkillCard
    virtual void useSkillCard(int cardIndex) = 0;
    virtual void applyMoveCard(int steps) = 0;
```

Penggunaan abstract base class IGameAction dalam tugas besar ini berfungsi sebagai jembatan antar komponen agar tidak saling bergantung secara langsung. Pendekatan ini digunakan untuk menghindari cirular dependency. Selain itu, pendekatan ini menerapkan Dependency Inversion Principle sehingga kelas seperti AuctionManager dan BankruptcyManager tidak bergantung pada kelas Game, tetapi pada abstraksi IGameAction. Dengan adanya kelas ini, semua interaksi cukup dilakukan pada satu interface tanpa perlu detail implementasinya.

b. Encapsulation

i. Kelas Tile

Cuplikan Kode Kelas Tile beserta Atribut dan Method

```
class Tile {
    protected:
        int index; // posisi tile dalam board
        std::string code; // kode unik
        std::string name; // nama
    public:
        // constructor
        Tile(int idx, std::string cd, std::string nm);
        // destructor
        virtual ~Tile() = default;
        // dipanggil saat player mendarat
        virtual void onLanded(Player& player, Game& game) = 0;
        // mendapatkan kode warna untuk tampilan board
        virtual std::string getDisplayColor();
        // getter atribut code
        std::string getCode() const;
        // getter atribut index
        int getIndex() const;
        // getter atribut nama
        std::string getName() const;
        // getter tile type
```

Atribut indeks, kode, dan nama pada kelas tile dienkapsulasi dengan modifier protected dan getter berupa method public. Enkapsulasi ini memungkinan pengaksesan langsung atribut-atribut Tile dari luar kelas tanpa pihak ketiga.

5. Bonus Yang dikerjakan

5.1. Bonus Kreasi Mandiri

Daftar kreasi tambahan:

1. Tampilan CLI yang berwarna dan interaktif dengan menggunakan ANSI color codes.
2. Menambahkan command “BANTUAN” yang menampilkan daftar lengkap seluruh perintah yang dapat digunakan oleh pengguna.

6. Pembagian Tugas

Modul (dalam poin spek)	Implementer	Tester
Save and Load	13524131	13524131
IGameAction + Game	13524138 13524131	13524138 13524131
ConfigLoader	13524131	13524131
CLIRenderer	13524131	13524131
Command Pattern	13524131	13524131
Tile	13524134	13524134
PropertyTile	13524134	13524134
ActionTile	13524134	13524134
SpecialTile	13524134	13524134

Dice	13524131	13524131
ActionCard	13524137	13524137
SkillCard	13524137	13524137
CardDeck	13524137	13524137
Player	13524138	13524138
AuctionManager	13524128	13524128
BankruptcyManager	13524128	13524128
TransactionLogger	13524131	13524131
Board	13524138	13524138
NimonspoliException	13524128	13524128

Lampiran

Lampiran A. Form Asistensi

https://drive.google.com/file/d/1CPl-fMvz3Nbgjc5OMa0FuGS2TL_lMYEp/view?usp=sharing